Forecasting Hourly $PM_{2.5}$ from Sparse Low-Cost Sensor Records in Uganda: Gap-Aware Imputation, Time-Aware Validation and Cloud Deployment

Charis Opol[1], **Cole Rodney Migisha**[1]
[1]Department of Computing and Technology, Faculty of Engineering, Design and Technology, Uganda Christian University, Mukono, Uganda (e-mail: b29654@students.ucu.ac.ug; b29657@students.ucu.ac.ug)

Corresponding author: Charis Opol (e-mail: b29654@students.ucu.ac.ug).

ABSTRACT Low-cost sensor networks such as AirQo have expanded $PM_{2.5}$ monitoring in Ugandan cities, but intermittent device availability leaves short, fragmented records, and it is unclear how well such records support forecasting. We develop and deploy a pipeline that predicts hourly $PM_{2.5}$ from 14,600 records of 149 AirQo sensors, which contain readings on only eight days in three bursts. PM_{10} was excluded as an input because it is a same-hour co-measurement of the same particles rather than a driver of $PM_{2.5}$, and is unavailable in advance. Extreme readings were replaced by the nearest normal value from the same sensor, and weather gaps were filled by windowed interpolation, time-of-day profiles or a BRITS-style bidirectional LSTM. Twelve models were compared under chronological hold-out, expanding-window and leave-sensors-out validation; SMOTE was tested and rejected. From weather, time and sensor identity alone, a tuned support vector regressor achieved a hold-out MAE of 9.34 μg/m^3 ($R^2 = 0.34$), against 17.60 μg/m^3 for a mean baseline; adding the previous three hours of $PM_{2.5}$ reduced the MAE to 8.43 μg/m^3 (LightGBM, $R^2 = 0.42$). The bidirectional imputer reduced imputation error by 21% but changed prediction error by only -0.4%. On a later full-month dataset (August 2026), the history-based models retained skill (MAE 13.70 against 19.21 μg/m^3), whereas the weather-only models performed close to the baseline, consistent with drift in six of nine input features. The forecast model is deployed as a cloud API with a dashboard. In sparse networks, recent pollutant history matters more than weather or sophisticated imputation.

INDEX TERMS $PM_{2.5}$ forecasting, Low-cost sensors, AirQo, Missing-data imputation, Time-series validation, Model deployment

I. INTRODUCTION

FINE particulate matter, written $PM_{2.5}$, is the mass of airborne particles smaller than 2.5 micrometres in diameter per cubic metre of air. Particles this small travel deep into the lungs, which is why $PM_{2.5}$ is the pollutant of greatest health concern in most cities. Its coarser relative, PM_{10}, counts all particles smaller than 10 micrometres, so every particle in $PM_{2.5}$ is also part of PM_{10}. The assignment brief cites a four-year reference-grade study (2018–2021) that found Kampala's annual mean $PM_{2.5}$ to be 39 μg/m^3, roughly eight times the World Health Organization (WHO) annual guideline of 5 μg/m^3 [1], with peaks in the dry seasons and during the morning and evening traffic hours.

Measuring $PM_{2.5}$ properly is expensive. A reference-grade instrument, such as a Beta Attenuation Monitor (BAM), which collects particles on a filter tape and weighs them by how much beta radiation they absorb, costs far more than most African cities can deploy widely. AirQo, based at Makerere University, fills that gap with low-cost sensors: small optical devices that estimate particle concentration from scattered light, built locally and placed across Ugandan cities [2], [3]. The price of that coverage is accuracy and continuity. Low-cost sensors drift with temperature and humidity, so their readings have to be calibrated, that is, statistically corrected against a reference instrument [4], and devices go offline for hours or days at a time.

Our task, set by the course brief, was to build a pipeline that takes raw AirQo sensor data all the way to a trained, evaluated, monitored and cloud-deployed model that predicts hourly $PM_{2.5}$. When we opened the data, one fact reshaped the project: although the file is labelled January to July 2026, it contains genuine readings on only eight days. This paper is therefore as much about what can be learned from a very short, fragmented record as it is about any one model.

The work went through two complete pipelines. The first used PM_{10} as an input and looked excellent on paper; ex-

amining why it looked excellent led us to rebuild it without PM_{10}. Along the way we tried several ideas that did not survive testing, and we report them, because they explain why the final pipeline looks the way it does. Our contributions are:

1) an honest account of the two pipelines, the alternatives we tried and the evidence behind each decision (Sections IV–V, Table 10);
2) a comparison of three gap-filling methods, including a bidirectional recurrent imputer, judged both on hidden values and on their effect on prediction (Section VI-B);
3) a comparison of twelve models under three validation setups and on a later, full-month dataset (Sections VI and VI-F);
4) drift monitoring with Evidently AI and a working cloud deployment, with the deployment routes that failed (Sections VII and VIII).

II. RELATED WORK

The AirQo network. Okure et al. [2] characterised ambient air quality in Ugandan urban areas with AirQo's low-cost sensors, and Sserunjogi et al. [3] published an hourly $PM_{2.5}$ and PM_{10} dataset from 68 AirQo monitoring stations for 2019–2020. Both describe the features we met in our data: uneven completeness and intermittent device availability.

How far low-cost sensors can be trusted. Adong et al. [4] calibrated AirQo sensors against a collocated BAM with ten machine-learning methods; a Random Forest lowered the $PM_{2.5}$ root mean squared error from 18.6 to 7.2 μg/m^3, and the calibration was rolled out to more than 120 devices. Lsoto et al. [5] compared 11 low-cost sensors with a BAM in Kampala over 13 months and found that the sensors followed the BAM at lower concentrations but spread out at higher values, and agreed better on 24-hour averages than hourly. Our data are AirQo's calibrated values, but these studies warn that hourly readings, especially high ones, still carry sensor error.

Weather and $PM_{2.5}$. Weather changes how particles form, grow, disperse and are washed out. Wang and Ogawa [6] found $PM_{2.5}$ negatively correlated with temperature in Nagasaki, with humidity effects that change direction at a threshold. Yang et al. [7] found that across Chinese cities humidity was positively correlated with $PM_{2.5}$ in North China and negatively in most other regions, and that the temperature relationship changed sign between autumn and winter. Closest to our setting, Wabinyai et al. [8] modelled daily $PM_{2.5}$ in Kampala and Fort Portal with meteorological variables and the previous day's $PM_{2.5}$; their models explained up to 57% and 80% of daily variability, and adding the lagged $PM_{2.5}$ improved accuracy significantly.

Machine-learning forecasts of $PM_{2.5}$. Febrian and Permai [9] forecast daily $PM_{2.5}$ in Jakarta and concluded that historical pollutant readings alone were sufficient for short-term forecasting, with meteorology adding little. Niyonzima et al. [10] forecast $PM_{2.5}$ one hour ahead on 759,480 hourly AirQo observations from 68 sites with an attention-based recurrent network, reaching a mean absolute error of 8.71 μg/m^3 and $R^2 = 0.68$, and also tested on held-out stations. Gürtepe et al. [11] predicted $PM_{2.5}$ regionally with gradient-boosted trees and reported a "station-blinded" validation, which scores the model at stations it never saw (RMSE 9.21 μg/m^3, against 8.28 μg/m^3 in their temporal evaluation).

Methods we borrowed. For gap filling we followed BRITS [12], a bidirectional recurrent imputer; for rare high-pollution hours we tested SMOTE [14] in its regression form [15], which has been used for $PM_{2.5}$ classes before [16]; and for explanations we used SHAP [17], [18].

The gap. The studies above work with months or years of data. None asks what can be done when a network's record is eight days long. That is the question this project forced on us.

TABLE 1. Missing and zero values in the raw file (14,600 rows).

Column	Empty	Zero	Total missing
Latitude	1	3,085	3,086
Longitude	0	3,067	3,067
Temperature	54	4,066	4,120
Humidity	206	4,159	4,365
$PM_{2.5}$	325	0	325
PM_{10}	–		430

III. DATA

A. WHAT THE FILE CONTAINS

The main file, `Sensor Data Jan – July 2026.csv`, holds 14,600 hourly records with 12 fields: site name, timestamp, sampling frequency, network (AirQo or AirGradient, a second low-cost sensor brand in the file), latitude, longitude, temperature, humidity, calibrated $PM_{2.5}$ and PM_{10}, site ID and device name. There are 149 devices at 145 named sites; some sites host more than one device, so the device, not the site, identifies a sensor. There are no duplicate device-hour pairs.

B. THREE SURPRISES

Only eight days. Readings exist on 1–3 January, 1–2 April and 1–3 June 2026, and nowhere else; the last reading is at 03:00 UTC on 3 June. That is 147 hours in which at least one sensor reported (Fig. 1).

Zeros that are not readings. The brief defines a 0 or an empty value as missing. Table 1 counts both. About 28.8% of AirQo-device rows report exactly 0 °C and 0% humidity together, in long unbroken blocks, and no AirGradient row does. A real 0 °C in Kampala is implausible; these are weather-sensor dropouts.

Readings that cannot all be right. $PM_{2.5}$ exceeded PM_{10} in 22.4% of rows, which is physically impossible since $PM_{2.5}$ is part of PM_{10}; 219 PM_{10} readings were below 1 μg/m^3; and 13 $PM_{2.5}$ readings were 200 μg/m^3 or more, the largest being 998.4 μg/m^3 at Nansana East Ward (Wakiso) with a PM_{10} of only 70.8 μg/m^3. These are the calibration problems the literature warns about [4], [5].

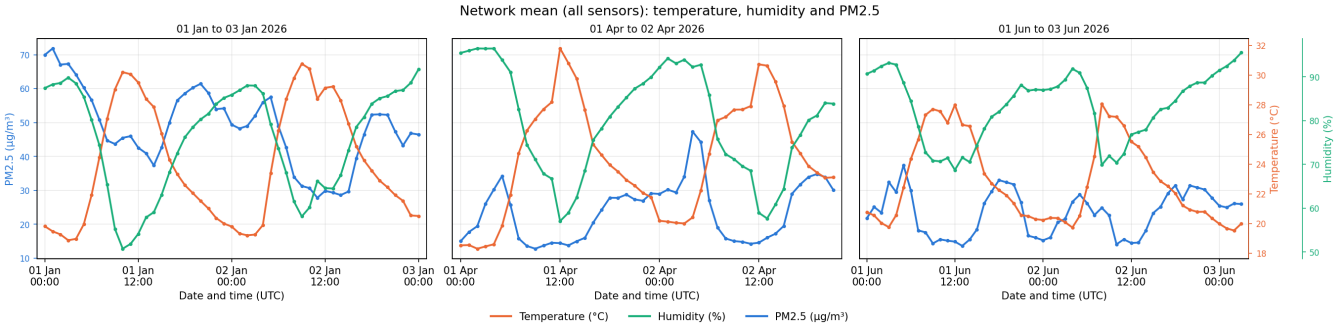


FIGURE 1. Network mean of temperature, humidity and $PM_{2.5}$. The time axis is cut into the only three windows that contain data.

TABLE 2. AQI categories of the brief and the number of raw $PM_{2.5}$ readings in each.

$PM_{2.5}$ ($\mu g/m^3$)	Category	Readings
0–9	Good	1,249
9.01–35.49	Moderate	8,494
35.5–55.49	Unhealthy for Sensitive Groups	1,820
55.5–125.49	Unhealthy	2,565
125.5–225.49	Very Unhealthy	137
above 225.49	Hazardous	10

Devices contribute between 20 and 147 records (median 105). Of the gaps between consecutive readings of a sensor, 13,668 are one hour and 783 are longer, up to about 150 days, the time between bursts.

C. THE AIR QUALITY INDEX CATEGORIES

The brief defines six Air Quality Index (AQI) categories, bands of $PM_{2.5}$ concentration with a health message attached (Table 2). We use them to colour calendars and maps and as a second, coarser measure of accuracy.

D. EXPLORATORY ANALYSIS

As the brief requires, the exploratory analysis focuses on five sensors inside Kampala District. We ranked single-device Kampala sites by the number of hours with $PM_{2.5}$, temperature and humidity all present, and accepted a site only if it had readings in every month that contains data, so that no panel would be empty: Nakasero II (`aq_66`, 147 complete hours), Lower Nsooba in Kawempe (`aq_g507`, 144), Salama Road (`ag_588c81267534`, 142), Namungona Primary School (`airqo_g5365`, 135) and Kabusu on Natete Road (`aq_g5_100`, 134).

Distributions are drawn as kernel density estimates (KDE) rather than histograms: a KDE replaces each reading with a small smooth bump and adds the bumps up, giving a smooth curve that does not depend on where histogram bin edges fall. $PM_{2.5}$ is strongly right-skewed, a few very high hours above a large Moderate mass, and data-poor sensors record somewhat higher $PM_{2.5}$ than data-rich ones. $PM_{2.5}$ peaks in the early morning and evening (Fig. 1). For the EDA we also expressed $PM_{2.5}$ as a Z-score using all observed readings

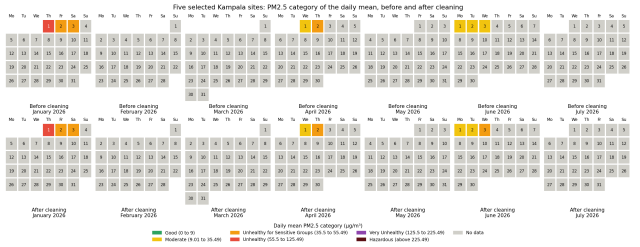


FIGURE 2. The five Kampala EDA sites: AQI category of the daily mean $PM_{2.5}$, before and after cleaning. Grey days have no data.

(mean 32.71, standard deviation 30.36 $\mu g/m^3$) to see how far individual hours stood from the norm. Fig. 2 shows the five sites as calendars coloured by AQI category.

Over all sensors, $PM_{2.5}$ is only weakly correlated with temperature (Pearson $r = -0.14$, Spearman -0.15) and humidity (0.09 and 0.10), while temperature and humidity are strongly inversely related (-0.81). Pearson's r measures a straight-line relationship; Spearman's measures whether two variables rise and fall together in rank order, whatever the shape. Within the five Kampala sites the links are stronger and consistent in sign: r from -0.13 to -0.51 for temperature and from 0.08 to 0.45 for humidity. That pattern, lower $PM_{2.5}$ in warm hours and higher in humid ones, agrees with the temperature findings of [6] and with the region-dependent humidity effect of [7]. Lagged cross-correlation, the correlation between one variable and another shifted by some hours, peaked at lag zero for temperature and humidity ($r = -0.88$); for temperature and $PM_{2.5}$ it was -0.24 at lag zero and -0.25 at one hour, so same-hour and one-hour-earlier weather carry the useful signal and longer lags only repeat the daily cycle. These are correlations, not evidence that weather causes $PM_{2.5}$ to change.

IV. PHASE 1: A FIRST PIPELINE, WITH PM_{10}

Our first pipeline followed the brief closely and used every measurement available, including PM_{10}. We describe it in the order we built it, including the ideas we later dropped.

A. CLEANING AND THE IDEAS WE TRIED

In our first pass we standardised the data types and left the zeros in place; we then adopted the brief's rule and treated zeros as missing. Timestamps were parsed as UTC, and `device_name` became the sensor key.

Coordinates. Missing coordinates were filled with the median of each site's own valid coordinates. Six sites never report a valid coordinate. We first tried k-nearest-neighbour (KNN) imputation for them: a KNN imputer fills a missing value from the k most similar records, here the five sites with the most similar temperature, humidity and pollution profile. We dropped it on reflection: two sites can have similar air without being close together, so the imputer would invent locations. The six sites are left without coordinates.

An hourly grid. Resampling each sensor onto a complete hourly grid from 1 January to 31 July (149 sensors $\times$ 5,088 hours $=$ 758,112 sensor-hours) made every missing hour explicit, with an `is_offline` flag. Interpolation, drawing a straight line between the readings on either side of a gap, was allowed only for the predictors (temperature, humidity and PM_{10}) and only across gaps of at most three hours. $PM_{2.5}$ was never interpolated, because it is the value being predicted.

Filling the long gaps. Curious whether the long gaps could be filled too, we applied a KNN imputer that matched each missing hour to hours with the same hour of day, weekday and month. The daily means of the result showed a suspiciously regular sawtooth. When we measured it, 96–98% of each EDA sensor's hourly series had become imputed, and only about 12% of its values were distinct: the imputer was copying the same few real readings into every matching hour. We reversed the decision and left long gaps missing.

B. MODELLING

The data were split chronologically: the model is trained on earlier hours and tested on later ones, as it would be used. Training covered 1 January–1 June (11,633 rows) and testing 2–3 June (2,642 rows). The target was converted to a Z-score, the number of standard deviations above the mean, using the training mean (34.49 μg/m^3) and standard deviation (31.70 μg/m^3) only, so that nothing about the test period leaked into training. Ridge, Linear, Polynomial, Random Forest, support vector and KNN regressors (Section V-G) were tuned with `TimeSeriesSplit`, a cross-validation scheme that always validates on hours later than those it trains on.

The tuned Random Forest was selected. On the test days it scored a mean absolute error (MAE) of 3.10 μg/m^3, a root mean squared error (RMSE) of 7.00 μg/m^3 and $R^2 = 0.898$ (these metrics are defined in Section V-H), and put 86.2% of test hours in the correct AQI category; the untuned Random Forest scored MAE 3.27 μg/m^3. We also asked whether the weather dropouts mattered: treating them as missing, keeping them as 0, or adding a dropout flag changed the test MAE only between 3.24 and 3.30 μg/m^3. The reason became clear from the feature importance: PM_{10} carried about 76% of the model's importance, temperature and humidity about 1% each.

C. PHASE 1 MONITORING AND FORECASTS

Evidently AI (Section VII) flagged 6 of the 7 columns it scored as drifted; those were the time features and the sensor identity, and temperature, humidity and PM_{10} were not among the scored columns. The drift reflected the make-up of the two windows, different months, weekdays and active sensors, rather than any change in the air. For the August forecasts, the unknown future weather and PM_{10} were replaced by each sensor's typical value for that hour of day.

D. THE TURNING POINT: WHY PM_{10} HAD TO GO

A 3.10 μg/m^3 error looked excellent, so we asked what the model was actually using. The answer was PM_{10}, and two problems followed.

First, PM_{10} is not a driver of $PM_{2.5}$. Because $PM_{2.5}$ is the fine part of PM_{10}, the two are measured from the same particles, by the same instrument, in the same hour, and they rise and fall together because they share sources, not because one causes the other. A model that uses PM_{10} learns this overlap, not how $PM_{2.5}$ comes about. The literature on what drives $PM_{2.5}$ points instead to emissions and to weather [6]–[8].

Second, PM_{10} is not known in advance, so a model that needs it cannot forecast. In the August forecast PM_{10} had to be replaced by a typical value, and the forecasts simply repeated each site's usual daily pattern.

After the feedback on our presentation we removed PM_{10} from every stage and rebuilt the pipeline. We expected, and accepted, a large drop in accuracy.

V. PHASE 2: THE REBUILT PIPELINE

Fig. 3 shows the rebuilt pipeline. It runs in one notebook, `train.ipynb`, from start to finish without manual steps. Everything that is fitted, meaning learned from data, such as hourly weather profiles, the imputer and the scalers, is fitted on the training part of each split only. Fitting on test data would be data leakage: information from the evaluation period seeping into training and making scores look better than they are.

A. CLEANING

PM_{10} is dropped immediately after loading. Zeros in temperature, humidity, $PM_{2.5}$, latitude and longitude become missing. A sensor would be removed if it had no valid reading or no valid $PM_{2.5}$ at all; none met that rule. Nine sensors report $PM_{2.5}$ but never report weather; we kept them and impute their weather.

B. EXTREME $PM_{2.5}$ VALUES

An outlier rule decides when a value is too far from the rest to trust. We used each sensor's own interquartile range (IQR), the spread of its middle 50% of readings: a reading is extreme when it lies above $Q_3 + 3\,\mathrm{IQR}$, where Q_3 is the sensor's 75th

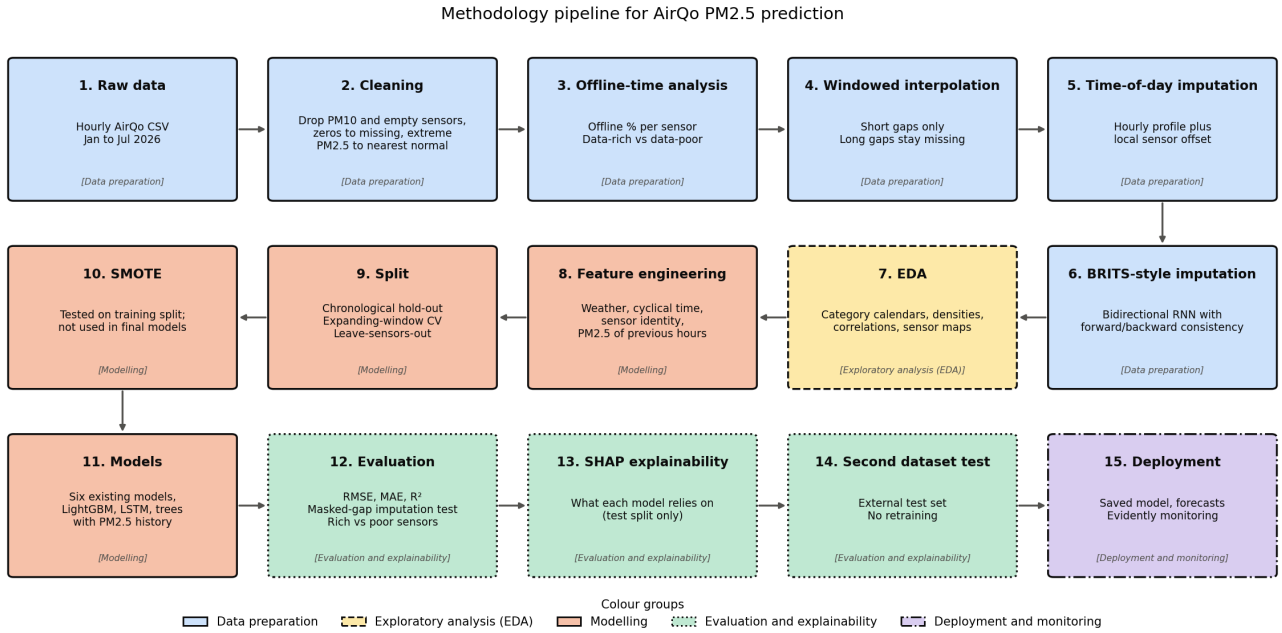


FIGURE 3. The Phase 2 pipeline. Colours and border styles mark the five groups of stages.

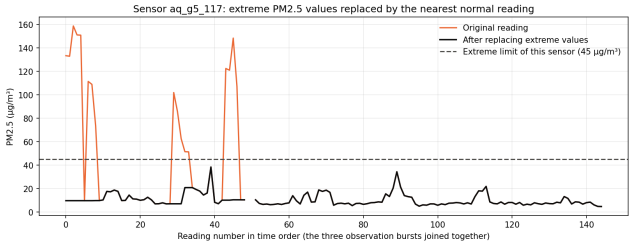


FIGURE 4. The sensor with the most extreme readings, before and after replacement by the nearest normal reading.

percentile. Rather than delete such a reading, we replace it with the nearest reading in time from the same sensor that is within the normal range, looking at most six hours away, and with the sensor's median if none exists.

The rule flagged 225 of 14,275 readings (1.5%) at 59 sensors; 223 were replaced by a normal reading at most five hours away (median one hour) and two by the sensor median. The mean $PM_{2.5}$ moved from 32.7 to 31.6 $\mu g/m^3$, the standard deviation from 30.4 to 26.8 $\mu g/m^3$ and the maximum from 998 to 415 $\mu g/m^3$ (Fig. 4). The original values are kept, and we also score the models against them. After the replacement the AQI counts of Table 2 become 1,270, 8,627, 1,820, 2,446, 107 and 5.

C. HOW OFTEN EACH SENSOR IS OFFLINE

A sensor is offline in an hour when it has no valid $PM_{2.5}$ reading. Over the whole January–July grid every sensor is 97.1–99.6% offline, which cannot separate sensors because the file holds only eight days. Over the 147 network-active hours it can, and we classify sensors on that basis (Table 3). We later ask whether models do worse on data-poor sensors.

TABLE 3. Sensor classes by offline time over the 147 network-active hours.

Class	Offline %	Sensors	Mean offline %
Data-rich	≤ 20	61	8.6
Intermediate	20–50	40	34.7
Data-poor	≥ 50	48	68.3

D. THREE WAYS OF FILLING WEATHER GAPS

Imputation means estimating a missing value. We imputed only the inputs (temperature and humidity); $PM_{2.5}$ targets are never imputed for training or scoring, because a model would then be scored against values another model invented. We built three methods, each a step up in ambition.

(a) Windowed interpolation. A gap is filled by a straight line only when it is at most three hours long, has real readings on both sides, and the 24-hour window centred on it holds at least six real readings. It filled 555 temperature and 551 humidity values; longer gaps stay empty.

(b) Time-of-day profile with a local offset. Weather follows a daily cycle, so for each sensor we computed the mean of each variable for every hour of the day, falling back to the sensor's network and then to all sensors when it had too few readings. A missing hour gets that profile value plus an offset measured from the sensor's own readings within 12 hours, which captures whether that day ran warmer or cooler than usual. For the models the profiles are refitted on training rows only.

(c) A BRITS-style bidirectional LSTM. One of us proposed a learned imputer, and we implemented one in the style of BRITS [12]. A recurrent neural network reads a sequence one step at a time and carries a memory, its hidden state, from step to step; a long short-term memory (LSTM) network [13] is a recurrent network with gates that decide what to keep and forget, which lets it remember over longer spans. A bidirectional model runs one LSTM forwards in time and one backwards, so each missing hour is estimated from both its past and its future. As in BRITS, each direction estimates every value before it sees it, its memory fades with the time since the last real reading, and a consistency loss, an extra training penalty when the two directions disagree, pulls their estimates together (weight 0.1). This is ordinary supervised learning with an added penalty, not reinforcement learning. Our version adds the hour of day and the sensor's offline share as always-known inputs, models $PM_{2.5}$ on a log scale, hides blocks of 1 to 12 hours during training so that the model practises on realistic gaps, and drops BRITS's classification part. It is written in TensorFlow and is retrained, with its scalers, for every split.

E. FEATURES

The model inputs are temperature and humidity; the hour of day, day of week and month; and the sensor identity. Time is cyclical: 23:00 is next to 00:00, and December next to January, which a plain number hides. We therefore encode each time variable as a sine and cosine pair, for example $\sin(2\pi\,\mathrm{hour}/24)$ and $\cos(2\pi\,\mathrm{hour}/24)$. The device name is one-hot encoded, turned into one yes/no column per sensor, so a model can learn a separate level for each sensor. Coordinates are not used, because the device already identifies the location.

For a second family of models we added $PM_{2.5}$ history: the same sensor's readings one, two and three hours earlier, their mean and how many of them exist. Only real earlier readings are used, so the value being predicted cannot appear in its own inputs. We call models with these features nowcast models, since they estimate the present from the recent past and need live readings, and models without them forecast models, since they can predict any future hour from the calendar and the weather.

F. AN EXPERIMENT WITH SMOTE

Unhealthy hours are rare in the data, and models tend to under-predict what they rarely see. SMOTE (Synthetic Minority Over-sampling Technique) [14] creates synthetic examples of a rare class by interpolating between real neighbours. Because $PM_{2.5}$ is continuous, we used its regression form [15]: training rows were grouped into $PM_{2.5}$ classes, rare classes were oversampled, and the target was interpolated along with the features. Only training rows were ever resampled.

It did not work. SMOTE made all eight models it was tried on worse on the hold-out at both strengths tested (Table 4). Oversampling the high-pollution hours raises the average $PM_{2.5}$ a model trains on; nothing in the inputs tells the model where those hours belong, so it raises its predictions everywhere, while the test days were cleaner than the training period (mean 23 against 33 μg/m^3). Cross-validation inside the training rows preferred a strength of 0.25, but that gain did not carry over to the hold-out. The final models use real rows only (`USE_SMOTE = False`); the experiment stays in the notebook.

TABLE 4. Hold-out MAE (μg/m^3) without SMOTE and with SMOTE at two strengths.

Model	None	0.25	0.5
Random Forest (untuned)	9.61	10.10	10.60
Random Forest (tuned)	9.63	10.12	10.73
KNN	10.01	10.41	10.84
LightGBM	9.82	28.67	23.39
SVR	9.34	30.22	28.02
Ridge	22.00	35.11	38.43
Linear / Polynomial	23.86	33.86	36.47

G. MODELS

We compared twelve models and a baseline:

- **Linear regression**, a weighted sum of the inputs; **Polynomial regression**, the same with squared and interaction terms; and **Ridge regression**, linear regression with a penalty on large weights that reduces overfitting.
- **Random Forest** [19], untuned and tuned: an average of many decision trees, each grown on a random sample of rows and features.
- **Support vector regression (SVR)**: a model that fits a smooth function and ignores errors smaller than a margin, using a kernel to capture non-linear patterns.
- **KNN regression**: the average of the k most similar training rows.
- **LightGBM** [20]: gradient boosting, in which each new tree is trained to correct the errors of the trees before it.
- **Two LSTMs** on six-hour windows of each sensor's series, one with weather and time and one that also sees the previous hour's $PM_{2.5}$ (32 units, dropout 0.2, early stopping with patience 8, meaning training stops after eight rounds without improvement on validation data).
- **Random Forest and LightGBM with $PM_{2.5}$ history**, the nowcast versions of the tree models.
- A **baseline** that always predicts the training mean. A model that cannot beat it has learned nothing useful.

Hyperparameters, the settings chosen before training such as the number of trees, were tuned by grid or random search with `TimeSeriesSplit` on the real training rows (three splits; two for SVR). Inputs were standardised inside each model's pipeline, and a value still missing at that point was filled with the training median. All tabular models use scikit-learn [21].

TABLE 5. The three validation setups.

Setup	Training and test rows
Chronological hold-out	Train 1 Jan–1 Jun (11,633 rows), test 2–3 Jun (2,642 rows)
Expanding-window CV	Test on 1 Apr, 2 Apr and 1 Jun in turn, training on everything earlier
Leave-sensors-out	Three folds, each holding out about 50 sensors entirely

H. VALIDATION SETUPS AND METRICS

One hold-out of two days is a thin basis for conclusions, so every model was scored three ways (Table 5). The expanding-window CV asks whether a result holds across periods; leave-sensors-out asks whether a model works at a sensor it has never seen, the same question as the station-blinded validation of [11].

The **mean absolute error** (MAE) is the average size of the error in μg/m^3, and is our selection metric because it is easy to explain. The **root mean squared error** (RMSE) squares errors before averaging, so it punishes large misses more. The **coefficient of determination**, R^2, is the share of the variation in $PM_{2.5}$ that the model explains: 1 is perfect, 0 is no better than predicting the mean, and negative is worse than the mean. We also report **AQI category accuracy**, the share of hours placed in the correct category. AUC and ROC curves are not used: they are measures for classifiers, and this is a regression problem.

VI. RESULTS

A. WHICH MODEL IS BEST

Table 6 gives all models under the three setups.

The nowcast models are best. LightGBM with $PM_{2.5}$ history has the lowest hold-out MAE (8.43 μg/m^3), and the Random Forest with history is the most consistent across setups (8.5, 10.3 and 9.0 μg/m^3). History lowered the hold-out MAE of the Random Forest by 11.6% and of LightGBM by 14.2%, and helped far more in the other setups, where the same models without history collapsed to about 27 μg/m^3 in the expanding-window CV. This matches Febrian and Permai's finding that pollutant history is enough for short-term forecasting [9], Wabinyai et al.'s finding that lagged $PM_{2.5}$ improves models in Kampala [8], and is close to the one-hour-ahead MAE of 8.71 μg/m^3 that Niyonzima et al. report on a far larger AirQo record [10], although our R^2 is lower.

Among the forecast models, the tuned SVR is best (MAE 9.34 μg/m^3, $R^2 = 0.34$). It is saved as `best_pm25_model.joblib` and makes the August forecasts; LightGBM with history is saved as `best_pm25_nowcast_model.joblib`. The forecast models beat the baseline on the hold-out but not in the expanding-window CV: trained on one or two bursts, they learn that month's level and carry it into the next. As in Gürtepe et al. [11], predicting at unseen sensors is harder than predicting later hours at known ones.

Against the untouched readings (53 of the 2,642 hold-out readings were replaced), the MAEs are 9.79 μg/m^3 for LightGBM with history and 10.62 μg/m^3 for the SVR. The SVR puts 75.2% of hold-out hours in the right AQI category and LightGBM with history 76.9%, but always predicting the training mean already scores 74.4%, because most hours are Moderate. Category accuracy is a weak yardstick on these data.

Fig. 5 shows the models following each sensor's level but not its sharp peaks. The mean error is close to zero for the SVR (-1.0 μg/m^3) and LightGBM with history (-0.5 μg/m^3). Every model is most accurate for data-rich and least accurate for data-poor sensors (SVR 8.73, 10.06 and 10.76 μg/m^3; LightGBM with history 7.66, 9.22 and 10.42 μg/m^3); history helps data-poor sensors less, because a sensor that is often offline often has no recent reading to use.

B. DOES BETTER IMPUTATION GIVE BETTER PREDICTIONS?

To measure imputation accuracy we needed the truth, so we ran a masked-gap test: we hid real readings in the hold-out period (one gap per sensor at a random position, three repeats), imputed them with each method, and compared. Fig. 6 and Table 7 give the results.

As an imputer, the bidirectional model is clearly better than the time-of-day method: its error is lower in all 15 variable-and-gap combinations (gaps of 1, 3, 6, 12 and 24 hours), by 21% on average, and about half the error for one-hour weather gaps. Interpolation is the most accurate for weather gaps of one to three hours, but it can fill only 5–12% of the values that are really missing (618 of 11,423 temperature values). The bidirectional model's fills are, however, smoother than real data: their standard deviation is 2.7 °C against 4.1 °C observed for temperature, and 11.9 against 26.8 μg/m^3 for $PM_{2.5}$. It is accurate on average because it predicts cautiously; it does not reproduce peaks.

The real question was whether this helps the predictions. We retrained five models on weather filled by each method, under all three setups, with the imputer fitted on training cells only and $PM_{2.5}$ withheld from it. Training on the bidirectional imputer's weather instead of the time-of-day weather changed the MAE by -0.4% on average and helped in 7 of 15 cases (Fig. 7). It helped the two LSTMs on the hold-out (10.89 to 9.48 μg/m^3 with history, -13%; 12.22 to 11.56 μg/m^3 without, -5%), hurt the SVR in the expanding-window CV (14.68 to 16.35 μg/m^3, $+11$%), and changed everything else by about 1% or less.

Decision. The final models keep interpolation followed by time-of-day imputation and are not retrained on the bidirectional imputer's values. There was no reliable gain; weather is a minor input for these models (Section VI-C); the one clear gain, for the LSTMs, leaves them behind the tree models with history, so the best and the deployed models would not change; and the bidirectional imputer must be

TABLE 6. Results under the three validation setups (mean over folds, μg/m^3), sorted by hold-out MAE. CV: expanding-window cross-validation.

	Chronological hold-out			CV	Leave-sensors-out	
Model	MAE	RMSE	R^2	MAE	MAE	R^2
LightGBM + $PM_{2.5}$ history	**8.43**	14.21	0.42	12.08	9.49	0.63
Random Forest + $PM_{2.5}$ history	8.50	14.25	0.41	10.25	9.03	0.65
SVR (tuned)	**9.34**	15.14	0.34	14.68	15.09	0.27
Random Forest (untuned)	9.61	16.14	0.25	27.02	14.90	0.22
Random Forest (tuned)	9.63	16.17	0.24	26.82	14.92	0.22
LightGBM (tuned)	9.82	16.08	0.25	27.45	15.13	0.27
KNN (tuned)	10.01	16.05	0.26	16.69	15.60	0.24
LSTM with $PM_{2.5}$ history	10.89	15.58	0.30	12.18	9.41	0.65
LSTM (weather and time)	12.22	18.53	0.01	20.70	15.61	0.26
Baseline (training mean)	17.60	21.17	−0.30	24.11	20.33	−0.03
Ridge (tuned)	22.00	25.37	−0.86	23.87	16.42	0.22
Linear (tuned)	23.86	27.87	−1.25	24.73	16.37	0.21
Polynomial (tuned)	23.86	27.87	−1.25	24.72	16.67	0.21

TABLE 7. Imputation MAE on hidden values. (a) does not fill gaps longer than three hours; BiLSTM: the bidirectional LSTM imputer.

Variable	Method	1 h	6 h	24 h
Temp. (°C)	(a) Interpolation	0.49	–	–
	(b) Time-of-day	1.61	1.87	2.13
	(c) BiLSTM	0.76	1.36	1.97
Humidity (%)	(a) Interpolation	1.80	–	–
	(b) Time-of-day	5.91	6.76	7.99
	(c) BiLSTM	2.90	5.66	7.38
$PM_{2.5}$ (μg/m^3)	(a) Interpolation	7.35	–	–
	(b) Time-of-day	9.10	10.42	12.13
	(c) BiLSTM	6.85	9.80	10.26

retrained for every split. Ridge, Linear, Polynomial, KNN and the Random Forests were not retested; because they use the same two weather inputs we expect the same result, but we did not measure it. The decision should be revisited if weather becomes a stronger input, if 6–12-hour gaps become common, or if the imputed series itself is the product.

C. WHAT THE MODELS RELY ON

SHAP (SHapley Additive exPlanations) [17] splits each prediction into a contribution from every input, borrowing an idea from game theory for sharing a payout fairly among players. We used TreeExplainer [18] for the tree models and GradientExplainer for the LSTM, on 400 test rows with 100 training rows as background, with the 149 sensor columns grouped into one feature.

Without history, the tree models rely on when and where a reading was taken, not on the weather (Table 8). The weight on month is a warning: with six training days in three months, the month mostly tells the model which burst a row came from, which will not transfer to other dates. With history, half of the attribution moves to the recent readings and the reliance on the calendar falls, which explains why those models are more stable. The extreme case shows the limit: the highest test reading (172 μg/m^3) is predicted as 33 μg/m^3. SHAP shows what a model uses, not what causes $PM_{2.5}$.

TABLE 8. Share of mean absolute SHAP value by feature group (test rows). RF: Random Forest; LGB: LightGBM; +H: with $PM_{2.5}$ history.

Feature group	RF	LGB	LGB+H
Recent $PM_{2.5}$	–	–	50%
Month	46%	32%	11%
Hour of day	18%	19%	12%
Sensor identity	18%	27%	11%
Day of week	5%	10%	7%
Temperature	8%	7%	7%
Humidity	5%	6%	3%

D. WHERE THE SENSORS ARE

Using the median coordinate of each sensor (143 of 149 have one), we mapped the network with `geopandas` and `shapely` and regional boundaries from geoBoundaries (Fig. 8). Prediction error does not follow sensor density: its rank correlation with the distance to the nearest neighbouring sensor is −0.08. It follows how polluted the site is (0.43) and, more weakly, how often the sensor is offline (0.22). The optional bonus map of sensors in the two worst categories at 13:00 on 1 August 2026 was not attempted.

E. FORECASTS FOR AUGUST 2026

The saved forecast model (SVR) produced daily forecasts for all 149 sensors for 1–14 August 2026 (2,086 sensor-days) and hourly forecasts for 1 August (3,576 sensor-hours). August lies outside the main data, so the time features come from the forecast timestamps, and temperature and humidity are each sensor's typical value for that hour of day, its climatology. All checks passed: correct row counts, every sensor and day, and no missing or negative predictions. Of the 2,086 sensor-days, 2,082 were forecast as Moderate and four as Unhealthy for Sensitive Groups. With typical weather as input, the forecasts can only repeat each sensor's usual pattern; the next subsection shows how far that is from what happened.

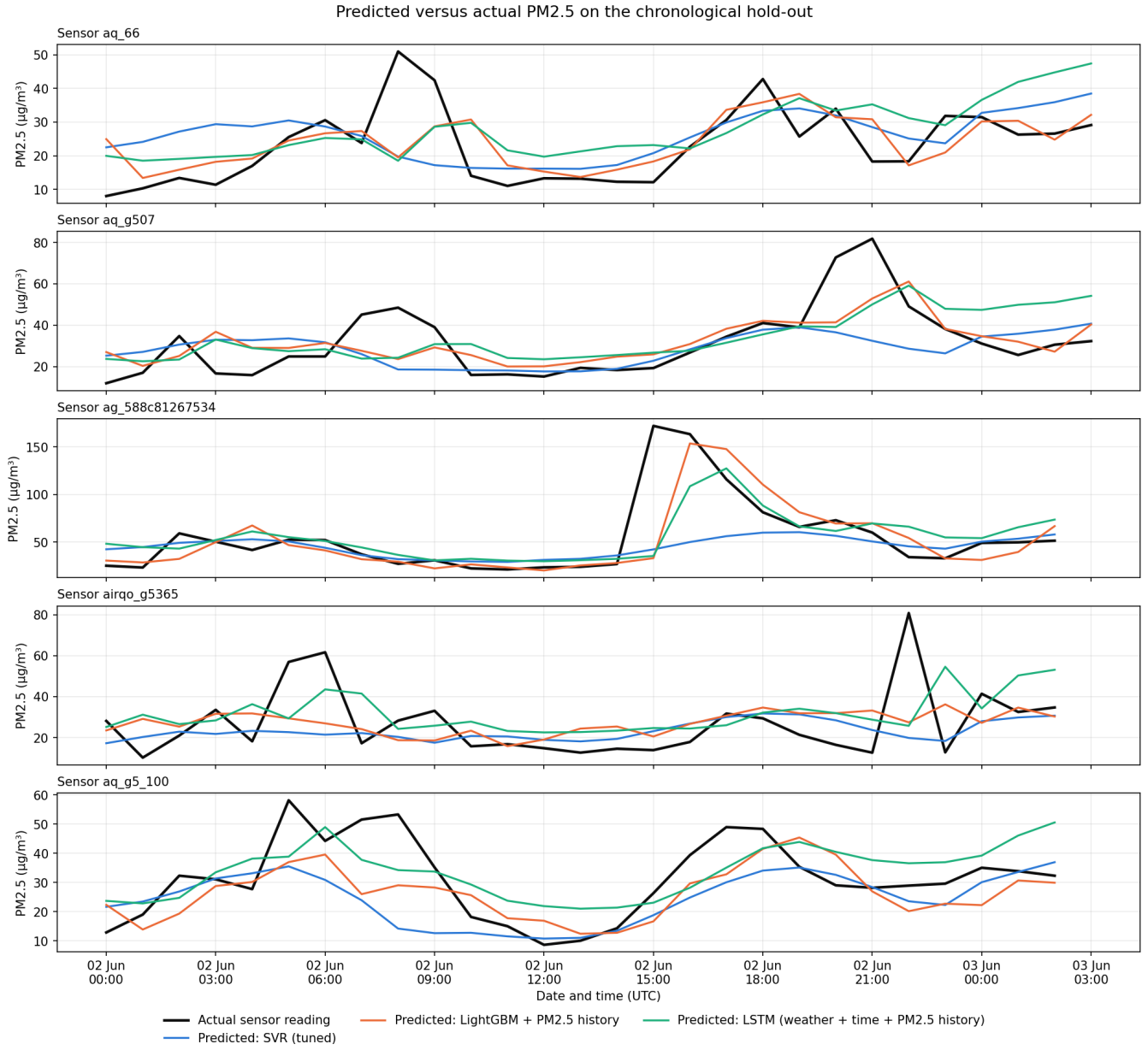


FIGURE 5. Predicted and actual $PM_{2.5}$ for the five Kampala EDA sensors on the hold-out days.

F. A SECOND DATASET: ALL OF AUGUST 2026

A later file, `AirQo_August_2026_hourly_matched.csv`, gave us something the main data could not: a full, continuous month. It has 73,274 hourly rows from 125 sensors for 1–31 August 2026, all of them also in the main data. We used it purely as a test set, with no retraining, after the same cleaning (its PM_{10} was dropped and its zero weather readings, about 14%, treated as missing). The extreme-value replacement was not applied to this file, so models are scored against its original readings. August was more polluted than the hold-out days (mean 35.5 against 23 $\mu g/m^3$).

The forecast models do not generalise to a new month: they are barely better than the baseline (Table 9), and the forecast model predicts Moderate on all 31 days while the actual daily mean was Unhealthy for Sensitive Groups on 16 of them (Fig. 9). The tree models with history keep part of their skill and match the category on 17 of 31 days. Unfamiliar sensors are not the explanation, since every sensor was seen in training; the models know these sensors only at their January, April and June levels. Missingness is not the cause either: 76 of the 125 sensors are data-rich in August. Imputation, by contrast, transfers well: the bidirectional imputer is as accurate on August temperature as on the hold-out (1.3 °C at six hours) and is the best $PM_{2.5}$ imputer at every gap length from six hours up. The daily cycle of weather carries over to a new month; $PM_{2.5}$ levels do not, unless the model is given

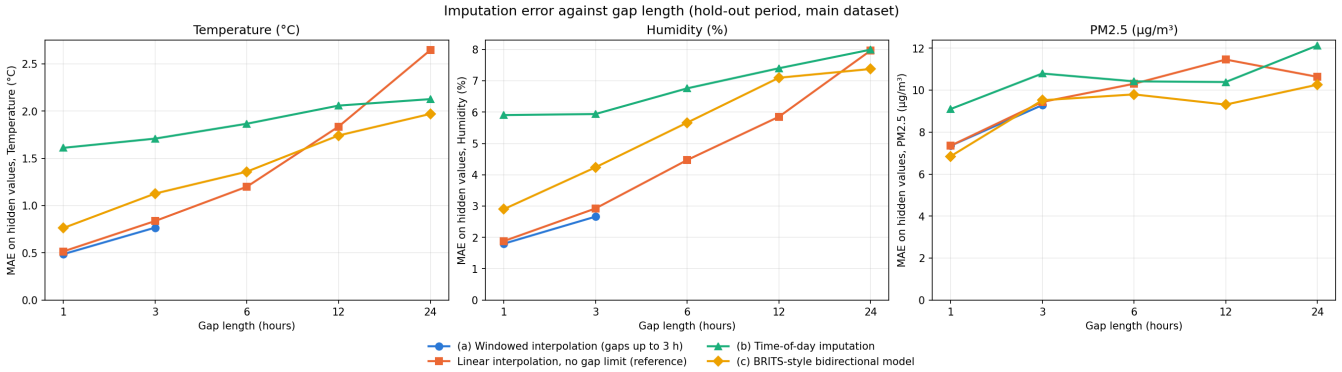


FIGURE 6. Imputation error against gap length for the three variables and the three methods.

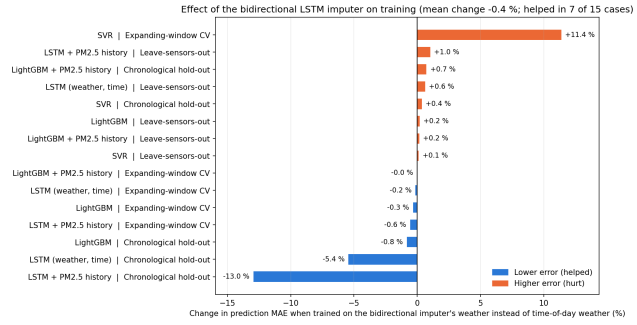


FIGURE 7. Change in prediction MAE when a model is trained on the bidirectional imputer's weather instead of the time-of-day weather. Negative values mean the bidirectional imputer helped.

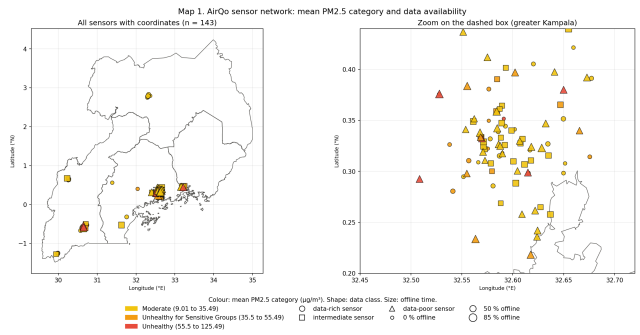


FIGURE 8. The sensor network: colour is the AQI category of the mean reading, shape the data class and size the offline time.

recent readings.

VII. MONITORING WITH EVIDENTLY AI

A deployed model can silently get worse when the data it receives change. Data drift is a change in the distribution of the inputs; model drift is a change in the model's performance. Evidently AI [22] (version 0.7.23) measures both by comparing a reference dataset, here the training period (11,633 rows), with a current dataset, the test period (2,642 rows). The Evidently cells are wrapped so that an Evidently error cannot stop the pipeline, as the brief requires.

TABLE 9. Prediction on the August dataset (μg/m^3), with the hold-out MAE for comparison.

Model	MAE	R^2	Hold-out MAE
Random Forest + history	13.70	0.30	8.50
LightGBM + history	14.02	0.27	8.43
SVR (forecast model)	18.32	−0.16	9.34
KNN	18.65	−0.13	10.01
LightGBM	18.79	−0.16	9.82
Random Forest (untuned)	19.05	−0.22	9.61
Baseline (training mean)	19.21	−0.01	17.60
LSTM with history	19.31	0.03	10.89
LSTM (weather and time)	19.47	−0.05	12.22

What Evidently reports. For each feature Evidently computes a drift score, a distance between the reference and current distributions. For numeric features it uses the normed Wasserstein distance, roughly how far the values would have to move to turn one distribution into the other, scaled by the reference spread; for categorical or low-cardinality features it uses the Jensen–Shannon distance, a symmetric measure of how different two sets of category shares are (0 for identical). A feature is flagged as drifted when its score exceeds the threshold of 0.1. The drift share is the fraction of features flagged, and the dataset as a whole is flagged as drifted when that share reaches 0.5.

Data drift. Eight of nine features drifted (share 0.89, Fig. 10). The largest scores were day of week (1.43 and 1.19 for the sine and cosine) and month (0.65), because the test window holds two weekdays of one month; temperature and humidity drifted moderately (0.36 and 0.37) and hour of day barely (0.08 and 0.15). On this problem the alarm means "the current data are not representative of the training data", not "pollution behaviour has changed".

Model drift. The selected forecast model's MAE rises from 5.90 μg/m^3 on the reference period ($R^2 = 0.81$) to 9.34 μg/m^3 on the current period ($R^2 = 0.34$). Part of that rise is expected, because reference predictions are made on the rows the model was trained on. The mean

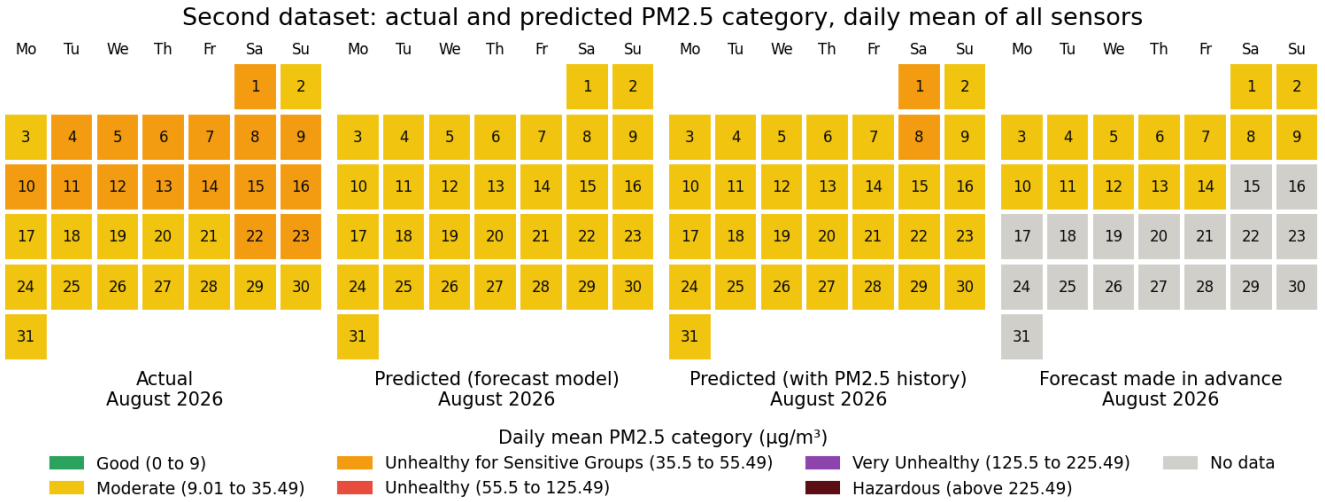


FIGURE 9. August 2026: actual and predicted AQI category of the daily mean, and the forecast made in advance for 1–14 August.

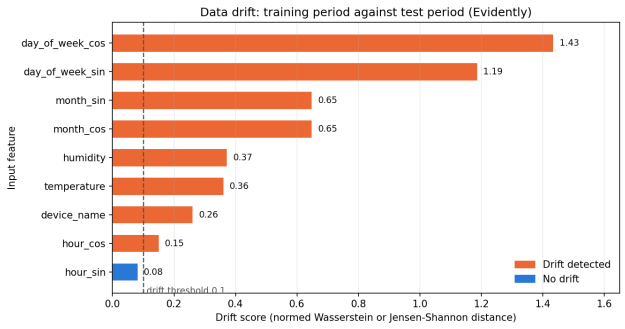


FIGURE 10. Evidently drift scores, training period against test period. Bars above the 0.1 threshold are flagged.

prediction (22.4 μg/m^3) stays close to the mean actual value (23.4 μg/m^3), so the model is not systematically biased. The mean error is -0.97 μg/m^3 with a standard deviation of 15.1 μg/m^3; the mean absolute percentage error (MAPE), the average error as a percentage of the true value, is 44.9%, and the largest single error is 134 μg/m^3: close to unbiased on average, but able to miss individual hours badly.

Drift against August. A second report compares the training data with August. Six of nine features drifted: month most (0.83), since August was never seen, then day of week (0.75 and 0.61), humidity (0.26) and the mix of sensors (0.25); temperature and hour of day did not drift. This is the input-side explanation of the weak August results. In production, a drift share this high together with rising error on newly labelled readings would be the signal to retrain.

VIII. DEPLOYMENT: THE ROUTES WE TRIED

Deployment took four attempts, and the failures shaped the final design.

Local verification first. Before any cloud attempt the pipeline was run end to end locally with the pinned versions in `requirements.txt` (Windows 11, Python 3.13, 16 logical CPUs, no GPU): 346 code cells with 0 errors and 70 figures, in 3,357 s (about 56 minutes), with a peak memory use of 1,230 MB. The forecast model file is 1.2 MB and the nowcast model 0.6 MB. Predicting one row takes about 14 ms and 3,576 rows about 1.4 s. A standalone inference script reproduced the notebook's predictions exactly on all 2,642 test rows. The model is saved with `joblib`, which serialises it: writes the fitted pipeline, its preprocessing, the target scaler and the feature lists into one file that can be loaded elsewhere.

Attempt 1: RunPod. The brief proposed RunPod, a pay-as-you-go cloud service that rents GPU machines ("Pods") by the hour, with a budget of $5. We prepared a run script for an RTX 2000 Ada Pod. RunPod requires credit to be bought in advance, which was financially straining for us, and this route did not succeed.

Attempt 2: Kaggle. We next tried to run the first, PM_{10}-based notebook end to end on Kaggle's free cloud notebooks as evidence of cloud execution. This attempt did not complete.

Attempt 3: Hugging Face. Hugging Face hosts machine-learning models and small web apps, called Spaces. We published the first model as the repository `Kyalis/airqo-pm25-random-forest` and its August forecasts as a Space, `Kyalis/airqo-pm25-forecasts`. Spaces that run Python on a server (Gradio, Docker or Streamlit) were refused with HTTP error 402 because they require a paid plan, so the Space is static: it shows stored forecasts and cannot run the model. The online repository still holds the first, PM_{10}-based model.

Attempt 4: Render and Vercel. For the rebuilt pipeline we wanted the model itself to answer requests. We split

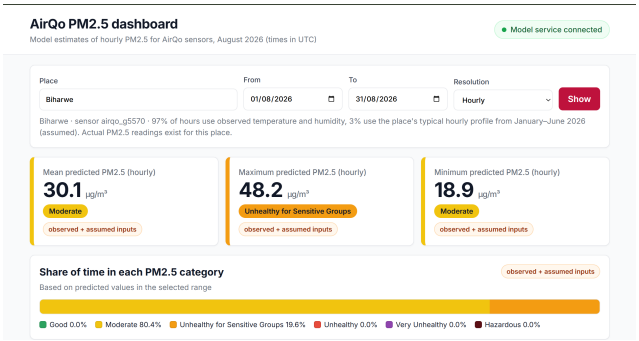


FIGURE 11. Dashboard: filters, summary of predicted $PM_{2.5}$ and share of time in each category.

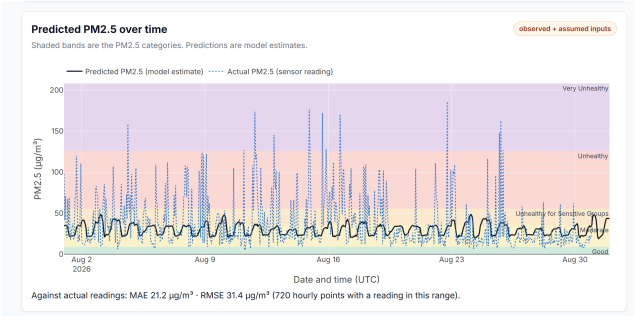


FIGURE 12. Dashboard: predicted $PM_{2.5}$ over the AQI bands, with the actual sensor readings and the error against them.

the problem in two. **Render** runs long-lived web services; it hosts an API (application programming interface), a web address that other programs can send requests to, around the forecast model at https://airqo-cole-charis-assgnmnt.onrender.com/. A client sends a JSON request to the `/predict` endpoint, the specific address that performs one action, with a place (sensor), a date and time in UTC, a temperature and a humidity; the service derives the cyclical time features from the timestamp exactly as in training, runs the serialised pipeline, converts the Z-score back to $\mu g/m^3$ and returns the predicted $PM_{2.5}$ with its AQI category. Only the packages needed to load the model and predict are installed (`numpy`, `pandas`, `scikit-learn`, `scipy`, `joblib`), which suits a free instance. **Vercel** hosts the front end, a dashboard at https://airqo-model.vercel.app/ that calls the Render service from the visitor's browser (Figs. 11–13).

The dashboard lets a user choose a place, a date range and an hourly or daily view; it shows the mean, maximum and minimum predicted $PM_{2.5}$, the predicted series over the AQI bands with the actual readings where they exist, a calendar, a map of the 143 places with coordinates, and a form that sends a request to the model and shows the exact request sent. It is honest about the model's limits: for the place in Fig. 12 the prediction stays between about 20 and 50 $\mu g/m^3$ while the actual readings spike above 150 $\mu g/m^3$, an MAE of 21.2 $\mu g/m^3$ against the August readings, in line with Section VI-F.

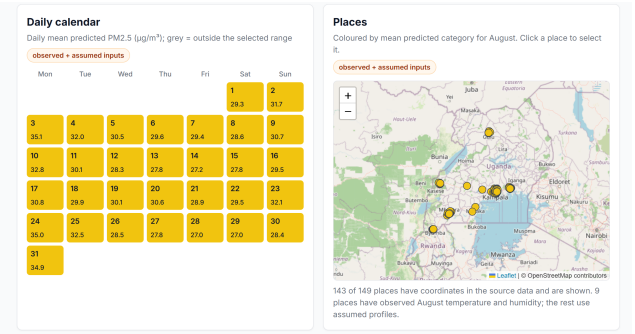


FIGURE 13. Dashboard: daily calendar of predicted $PM_{2.5}$ and map of the places.

The service runs the forecast model only. The more accurate nowcast model is saved but not served, because every request would need the sensor's readings from the previous three hours. A free Render instance sleeps when idle and is slow on the first request, and the service loads a fixed model file, so retraining still means re-running the notebook and redeploying.

IX. DISCUSSION

Table 10 summarises the route from the first pipeline to the final one. Two lessons stand out. The first is that an excellent score deserves suspicion: the Phase 1 model's MAE of 3.10 $\mu g/m^3$ came from a co-measurement of the target, and only by asking what the model relied on did we see that it could not forecast. The second is that, on records this short, the simplest strong signal wins. Recent $PM_{2.5}$ was worth more than any amount of care over the weather inputs: the most accurate imputer we built changed prediction error by less than half a percent, while three hours of history cut it by 12–14% on the hold-out and kept the models useful on a new month.

Limitations.

- **Eight days of data.** Three two-day bursts cannot show seasonal patterns, and the two-day hold-out is short; the three setups and the August test are needed to judge stability.
- **Weak predictors without PM_{10}.** Temperature and humidity carry only 3–8% of the SHAP attribution here. Eight days are too few for a model to learn the weather effects described in the literature [6]–[8].
- **Sensor accuracy.** The impossible $PM_{2.5}$>PM_{10} readings point to calibration error, which the models learn as it is [4], [5].
- **Extreme values.** Replacing them lowers the reported errors (about 1.3 $\mu g/m^3$ against the untouched readings), and the replacement was not applied to the August file.
- **The imputation decision** was measured for five models only.
- **Deployment.** The required RunPod run with logs was not achieved; the live service runs the forecast model on free tiers, and the published Hugging Face model is still

TABLE 10. Decision log: what we tried, what we chose and why.

Decision	Alternative tried or considered	Evidence	Outcome
Treat zeros as missing	Keep zeros as values (first pass)	The brief defines 0 as missing; 0 °C/0% occur in long dropout blocks	Missing, then imputed
Leave six sites without coordinates	KNN on weather and pollution profiles	Similar air does not imply nearby sites	Left missing
Leave long $PM_{2.5}$ gaps missing	KNN filling of long gaps	96–98% of series imputed, ≈12% distinct values, artificial sawtooth	Rejected
Remove PM_{10}	Keep PM_{10} as an input (Phase 1)	Co-measurement of the same particles, not a driver; 76% of importance; unknown in advance	Dropped everywhere
Replace extreme $PM_{2.5}$	Keep or delete them	225 readings (1.5%) above $Q_3 + 3$ IQR; maximum 998 μg/m^3	Nearest normal reading; originals kept
No SMOTE	SMOTE for regression at 0.25 and 0.5	All eight models worse on the hold-out	`USE_SMOTE = False`
Time-of-day imputation for training	BRITS-style bidirectional LSTM	21% better imputer, but -0.4% change in prediction MAE over 15 cases	Time-of-day kept
Two saved models	One model	Forecasts need no live readings; nowcasts are more accurate	SVR and LightGBM + history
Render + Vercel	RunPod, Kaggle, Hugging Face Spaces	RunPod credit too costly; Kaggle run did not complete; server-side Spaces need a paid plan	Render API + Vercel dashboard

the Phase 1 version.

Future work. The largest gain would come from training on continuous data, such as the August file. Beyond that: combine lagged pollutant features with multi-step forecasting and forecast weather; use same-hour $PM_{2.5}$ from neighbouring sensors, which are dense around Kampala; move the service to a managed cloud platform such as Google Cloud (Cloud Run or a Vertex AI endpoint) or Azure (Azure Machine Learning online endpoints or Container Apps) for autoscaling and model versioning; export the model to ONNX, an open format that lets a model run in a web browser, so a static page could run it; and automate monitoring, with scheduled Evidently reports, retraining when error grows and rollback to the previous model if a retrained one validates worse.

X. CONCLUSION

We built and deployed a reproducible pipeline that predicts hourly $PM_{2.5}$ from AirQo sensor data containing only eight days of readings. Our first model looked excellent because it used PM_{10}, a same-hour co-measurement of the target; removing it gave an honest forecast model, a tuned SVR with a hold-out MAE of 9.34 μg/m^3, and nowcast models with three hours of $PM_{2.5}$ history that reached 8.43 μg/m^3 and stayed useful on a later, full month. We tried and rejected KNN-invented coordinates and long-gap fills, SMOTE oversampling and, for training, our most accurate imputer, each on measured evidence. The model is served by an API on Render with a dashboard on Vercel after RunPod and Kaggle attempts failed and Hugging Face could host only a static page. In a sparse network, recent pollutant history matters more than weather or sophisticated imputation; more continuous data, not more complex models, is the next step.

USEFUL LINKS

Code (GitHub):
https://github.com/coleRodneyMigisha/airqo_cole_charis_assgnmnt

Dashboard (frontend):
https://airqo-model.vercel.app/

REFERENCES

[1] World Health Organization, WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide. Geneva, Switzerland: WHO, 2021, ISBN 978-92-4-003422-8.

[2] D. Okure, J. Ssematimba, R. Sserunjogi, N. L. Gracia, M. E. Soppelsa, and E. Bainomugisha, "Characterization of ambient air quality in selected urban areas in Uganda using low-cost sensing and measurement technologies," Environ. Sci. Technol., vol. 56, no. 6, pp. 3324–3339, 2022, doi: 10.1021/acs.est.1c01443.

[3] R. Sserunjogi, J. Ssematimba, D. Okure, D. Ogenrwot, P. Adong, L. Muyama, N. Nsimbe, M. Bbaale, and E. Bainomugisha, "Seeing the air in detail: Hyperlocal air quality dataset collected from spatially distributed AirQo network," Data in Brief, vol. 44, Art. no. 108512, 2022, doi: 10.1016/j.dib.2022.108512.

[4] P. Adong, E. Bainomugisha, D. Okure, and R. Sserunjogi, "Applying machine learning for large scale field calibration of low-cost PM2.5 and PM10 air pollution sensors," Applied AI Letters, vol. 3, no. 3, 2022, doi: 10.1002/ail2.76.

[5] D. Lsoto, R. Sserunjogi, G. Okello, Y. Chen, J. A. Patz, and J. J. Schauer, "A framework for interpreting air pollution sensor data: Bridging gaps in monitoring, analysis and community action," J. Air Waste Manage. Assoc., vol. 76, no. 6, pp. 472–483, 2026, doi: 10.1080/10962247.2026.2635079.

[6] J. Wang and S. Ogawa, "Effects of meteorological conditions on PM2.5 concentrations in Nagasaki, Japan," Int. J. Environ. Res. Public Health, vol. 12, no. 8, pp. 9089–9101, 2015, doi: 10.3390/ijerph120809089.

[7] Q. Yang, Q. Yuan, T. Li, H. Shen, and L. Zhang, "The relationships between PM2.5 and meteorological factors in China: Seasonal and regional variations," Int. J. Environ. Res. Public Health, vol. 14, no. 12, Art. no. 1510, 2017, doi: 10.3390/ijerph14121510.

[8] F. R. Wabinyai, R. Sserunjogi, G. Lubisia, D. Okure, E. Akugizibwe, J. Kutesakwe, A. Nshimye, A. Ndyabakira, and E. Bainomugisha, "Multilinear regression analysis of PM2.5 in Kampala and Fort Portal cities: Effects of meteorological factors and lagged pollution," Atmos. Environ. X, vol. 29, Art. no. 100411, 2026, doi: 10.1016/j.aeaoa.2025.100411.

[9] S. Febrian and S. D. Permai, "PM2.5 index prediction in Central Jakarta using pollution and meteorological data with CNN-LSTM," E3S Web Conf., vol. 680, Art. no. 00067, 2025, doi: 10.1051/e3sconf/202568000067.
[10] I. Niyonzima, R. W. Mwangi, and P. M. Muriithi, "Nonlinear optimization of recurrent neural networks in the prediction of air quality," Discover Artif. Intell., vol. 6, no. 1, 2026, doi: 10.1007/s44163-026-01623-3.
[11] İ. Ç. Gürtepe, İ. T. Şenkal, A. Ünal, G. Güllü, Y. Aslanoğlu, and J. D. Marshall, "Machine learning-driven regional prediction of PM2.5 concentrations in the eastern Mediterranean bridging spatial data gaps in air quality monitoring," Environ. Model. Softw., vol. 192, Art. no. 106586, 2025, doi: 10.1016/j.envsoft.2025.106586.
[12] W. Cao, D. Wang, J. Li, H. Zhou, L. Li, and Y. Li, "BRITS: Bidirectional recurrent imputation for time series," in Advances in Neural Information Processing Systems 31 (NeurIPS 2018), 2018, arXiv:1805.10572.
[13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Comput., vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
[14] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002, doi: 10.1613/jair.953.
[15] L. Torgo, R. P. Ribeiro, B. Pfahringer, and P. Branco, "SMOTE for regression," in Progress in Artificial Intelligence (EPIA 2013), Lecture Notes in Computer Science. Berlin, Germany: Springer, 2013, pp. 378–389, doi: 10.1007/978-3-642-40669-0_33.
[16] S. Saminathan and C. Malathy, "Ensemble-based classification approach for PM2.5 concentration forecasting using meteorological data," Front. Big Data, vol. 6, Art. no. 1175259, 2023, doi: 10.3389/fdata.2023.1175259.
[17] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems 30 (NIPS 2017), 2017, arXiv:1705.07874.
[18] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," Nat. Mach. Intell., vol. 2, no. 1, pp. 56–67, 2020, doi: 10.1038/s42256-019-0138-9.
[19] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.
[20] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems 30 (NIPS 2017), 2017.
[21] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.
[22] Evidently AI, "Evidently: Open-source ML and data monitoring," version 0.7.23. [Online]. Available: https://github.com/evidentlyai/evidently